E2P: Exploração Inicial de Aplicações Web Orientada por Ações com um Modelo de Linguagem Local

Henrique Oliveira da Cunha Franco[1], **Humberto Torres Marques Neto**[1]

[1]Instituto de Informática – Pontifícia Universidade Católica de Minas Gerais (PUC Minas)

henrique.franco.1448652@sga.pucminas.br, humberto@pucminas.br

Abstract. *This work investigates how far a local language model can conduct initial exploration of an unfamiliar web application and observe potential failures. E2P connects project inspection, a bounded action catalogue, state observations, test generation, and evidence review. The evaluation combines educational applications and a component-based task application through repeated explorations and browser checks. Without a supplied navigation script or defect description, the model initiates data-entry, persistence, and filtering operations and reaches states with reproducible application exceptions. Controlled checks show that compiled tests distinguish reference behavior from altered implementations and preserve the approved expected value when it differs from the recorded observation. A form-validation case demonstrates a reproducible observation associated with an unjustified defect hypothesis. The findings characterize initial exploration and traceable verification under the evaluated quantization, context allocation, and inference budget.*

Resumo. *Este trabalho investiga até que ponto um modelo de linguagem local consegue realizar uma exploração inicial de uma aplicação web desconhecida e observar possíveis falhas. O E2P conecta inspeção do projeto, catálogo limitado de ações, observações de estado, geração de testes e revisão de evidências. A avaliação combina aplicações didáticas e uma aplicação de tarefas baseada em componentes, por meio de explorações repetidas e verificações de navegador. Sem roteiro de navegação ou descrição de defeito fornecidos, o modelo inicia operações de entrada, persistência e filtragem e alcança estados com exceções reproduzíveis. As verificações controladas mostram que os testes compilados distinguem a referência de implementações alteradas e preservam o valor esperado aprovado quando ele difere da observação registrada. Um caso de formulário demonstra uma observação reproduzível associada a uma hipótese de defeito sem justificativa. Os resultados caracterizam a exploração inicial e sua verificação rastreável sob a quantização, a alocação de contexto e o orçamento de inferência avaliados.*

1. Introdução

Testes ponta a ponta (*End-to-End*, E2E) exercitam jornadas de uso sobre um sistema integrado. Sua utilidade depende não apenas de executar interações, mas também de selecionar comportamentos relevantes e verificar seus resultados. Para uma aplicação

web sem uma rotina de garantia de qualidade (*Quality Assurance*, QA), preparar essa automação envolve entender o produto, configurar seu ambiente, explorar suas funções, especificar casos e interpretar falhas. Automatizar somente a escrita de scripts deixa parte importante desse trabalho em aberto.

Modelos de linguagem de grande porte (*Large Language Models*, LLMs) têm sido investigados em geração de testes, reprodução de defeitos e apoio à manutenção de software [Wang et al. 2024, Kang et al. 2023]. Entretanto, uma resposta textual plausível não comprova que controles existem, que uma jornada foi executada ou que uma expectativa é correta. Estudos de agentes web evidenciam dificuldades na conclusão de tarefas realistas [Zhou et al. 2024, Drouin et al. 2024]. Em QA, essas dificuldades se somam ao risco de transformar um comportamento normal em relato de defeito.

Neste trabalho, *orientado por ações* caracteriza o papel do modelo na orquestração: a partir da observação da aplicação, ele escolhe uma operação em um catálogo explícito, executada e verificada por ferramentas. O protótipo Action E2E Prototype (E2P) integra um modelo existente a documentação e trechos de código, estado estruturado da interface, percepção visual opcional, ações limitadas e recursos de revisão humana. Seu propósito é apoiar a preparação de testes e a análise de evidências em aplicações web. A avaliação utiliza aplicações de pequeno porte com funções identificáveis, permitindo examinar como as decisões do modelo se relacionam com o comportamento observado.

O problema de pesquisa é: *até que ponto um modelo de IA consegue realizar uma exploração inicial de uma aplicação web desconhecida e, nesse processo, observar possíveis falhas?* O objetivo geral é investigar essa capacidade por meio do E2P, que conecta decisões do modelo à execução real da aplicação. Os objetivos específicos são: (i) reconhecer funções a partir de documentação, código e interface; (ii) selecionar e executar ações relacionadas ao propósito do produto; (iii) registrar estados e evidências que permitam revisar a trajetória; e (iv) analisar hipóteses de falha e os mecanismos utilizados para verificá-las.

A contribuição é o E2P, um instrumento local que torna rastreável o percurso entre decisão do modelo, ação executada, evidência e asserção. As trajetórias registradas mostram que o modelo inicia operações relacionadas à finalidade da aplicação sem receber um roteiro e alcança estados nos quais a instrumentação captura falhas reproduzíveis. Os ensaios de navegador demonstram que os testes compilados distinguem as propriedades de referência de alterações controladas e conservam a expectativa aprovada mesmo quando ela difere do observado. Esses resultados articulam a iniciativa exploratória com a verificação do que foi efetivamente alcançado.

O restante do artigo apresenta a fundamentação e os trabalhos relacionados na Seção 2, a metodologia na Seção 3, a implementação na Seção 4, os resultados na Seção 5, as ameaças à validade na Seção 6 e as conclusões na Seção 7.

2. Referencial Teórico e Trabalhos Relacionados

2.1. Da execução de interações à avaliação de comportamento

Um caso de teste associa condições iniciais, ações e resultados esperados. Um oráculo é o critério utilizado para decidir se o resultado observado é aceitável. No E2P, critérios em linguagem natural antecedem a automação e podem ser revisados. Essa separação é

necessária porque a validade sintática do script, a existência do seletor e a aprovação da execução não garantem correspondência com a intenção do caso.

A revisão de Wang et al. [Wang et al. 2024] organiza aplicações de LLMs em diferentes atividades de teste e destaca desafios de sua integração com outras técnicas. A distinção entre gerar entradas, produzir asserções e avaliar resultados orienta este estudo: uma melhoria em uma dessas atividades não é automaticamente transferível às demais. Em particular, medir somente quantos testes passam favorece testes permissivos, que podem permanecer verdes mesmo diante de um defeito.

Na avaliação de hipóteses, distinguem-se três afirmações: uma interação ocorreu; seu efeito foi reproduzido; esse efeito viola uma expectativa justificável. As duas primeiras são observacionais. A terceira requer evidência normativa, como um requisito, uma mensagem explícita da interface ou uma regra de domínio sustentada pelo projeto. O código auxilia na explicação causal, mas não constitui sozinho uma especificação infalível: ele também pode conter o defeito investigado. Essa distinção fundamenta a análise dos falsos positivos apresentada adiante.

2.2. Agentes de interface e testes baseados em uso

WebArena oferece aplicações e tarefas web para avaliar conclusão funcional, em vez de apenas a plausibilidade do texto produzido, e compara agentes com participantes humanos [Zhou et al. 2024]. WorkArena e BrowserGym enfatizam ações padronizadas, observações multimodais e tarefas de software corporativo [Drouin et al. 2024]. Esses trabalhos fundamentam o uso de ambientes registráveis e medidas ligadas ao comportamento efetivamente realizado. O E2P investiga a iniciativa exploratória anterior à definição de uma tarefa de teste, preservando a trajetória que poderá fundamentá-la.

No domínio móvel, GPTDroid estrutura a exploração como interação iterativa entre modelo e aplicação, utilizando informações da interface, retorno da execução e memória de funcionalidades [Liu et al. 2024]. O E2P adota o ciclo de observar e agir em aplicações web, restringindo decisões a ações catalogadas. Sua memória de revisões humanas recupera informações entre execuções, complementando a trajetória da sessão atual.

Avgust transforma vídeos de uso em uma representação de estados para apoiar a geração de testes em aplicações móveis [Zhao et al. 2022]. A aproximação com o E2P está na valorização de jornadas observadas. A diferença é a origem dessas jornadas: no E2P, elas são produzidas pela exploração guiada pelo modelo, e não fornecidas previamente como vídeos de usuários. Isso reduz a necessidade de uma gravação inicial, mas transfere ao explorador a responsabilidade de alcançar comportamentos úteis.

2.3. Geração e verificação de requisitos

LIBRO gera testes de reprodução a partir de relatos de defeitos e utiliza pós-processamento para avaliar e ordenar candidatos [Kang et al. 2023]. Enquanto esse trabalho parte de um problema relatado, o E2P também propõe hipóteses a partir da interface explorada. Portanto, além da reprodução, precisa avaliar se a própria alegação de erro é válida.

Ferreira et al. [Ferreira et al. 2025] apresentam um estudo industrial com geração de cenários Gherkin a partir de histórias de usuário e conversão para testes Cypress

usando HTML. Os cenários foram considerados úteis em 95% das avaliações; 60% dos testes puderam ser utilizados como gerados, enquanto os demais exigiram ajustes, nova geração ou descarte. Esse resultado apoia o uso de artefatos intermediários revisáveis. Diferentemente daquele estudo, o E2P precisa inferir oportunidades de teste de um projeto e de sua execução, sem pressupor um conjunto completo de histórias de usuário.

Panta combina análise estática de fluxo e cobertura dinâmica para orientar iterativamente a geração de testes unitários [Gu et al. 2026]. TestWeaver usa recortes de programa, testes relacionados e anotações de execução para fornecer contexto dirigido ao modelo [Le et al. 2026]. Essas abordagens motivam o uso de sinais verificáveis e de contexto selecionado. No E2P, a análise se concentra nas funções de interface exercitadas e em seus resultados observáveis.

GUISpector verifica requisitos em linguagem natural sobre protótipos de interface com um agente multimodal e disponibiliza resultados para supervisão [Kolthoff et al. 2025]. O E2P compartilha o interesse na rastreabilidade entre requisito e trajetória, mas também investiga a descoberta dos próprios casos. Isso torna a procedência da expectativa um problema central: inferir um requisito exige cautela adicional em relação a verificar um requisito fornecido.

SpecOps decompõe o teste de agentes de interface gráfica (*Graphical User Interface*, GUI) em geração, preparação, execução e validação [Ahmed et al. 2026]. Seu objeto são agentes de IA em ambientes reais, enquanto o E2P avalia aplicações web. A separação dos estágios orienta a arquitetura do protótipo e a identificação da origem de cada evidência.

2.4. Posicionamento do trabalho

O E2P se posiciona na investigação da exploração inicial de aplicações web por um modelo de IA orientado por ações. Diante de um projeto apresentado sem roteiro de teste, o modelo precisa reconhecer sua finalidade, identificar possibilidades de interação e escolher como obter informação sobre seu funcionamento. A questão central é quanto dessa atividade pode ser realizada de forma útil e que indícios de falha podem surgir durante o percurso.

Os trabalhos relacionados oferecem fundamentos para essa investigação: agentes de interface sustentam o ciclo de observar e agir; geração de cenários inspira os artefatos intermediários; e estudos de reprodução e avaliação orientam a conferência das hipóteses. Essas contribuições informam a arquitetura do E2P e os critérios de análise. O objeto empírico deste trabalho é a atuação do modelo dentro dessa implementação, conectada a uma aplicação executada e a evidências de suas próprias decisões.

3. Metodologia

3.1. Delineamento e questões de pesquisa

A pesquisa é aplicada e exploratória, centrada na atuação do modelo dentro do E2P. A inspeção do projeto e a observação da interface fornecem o contexto para escolher ações, acompanhar consequências e propor verificações e hipóteses.

As questões de pesquisa são: **QP1**, quais funcionalidades o modelo reconhece e consegue exercitar durante uma exploração inicial; **QP2**, quais evidências permitem

identificar possíveis falhas da aplicação; **QP3**, como os mecanismos de validação do E2P preservam a relação entre ação, resultado e expectativa; e **QP4**, como as condições de execução e os recursos computacionais delimitam a profundidade alcançada. O foco está na instância atual do protótipo e nas atividades que ela torna executáveis.

A avaliação articula três fontes de evidência: caracterização das aplicações e inspeção de seus artefatos; novas explorações com configuração mantida fixa; e verificações de navegador sobre propriedades determinadas. A trajetória produzida pelo modelo é analisada separadamente da propriedade exigida por um teste de regressão. Essa organização permite observar a iniciativa exploratória e conferir os mecanismos que sustentam sua interpretação.

3.2. Aplicações utilizadas e comportamentos de interesse

Para observar decisões em interfaces com finalidades diferentes, foram utilizados os exemplos públicos de Vanilla Web Projects [Traversy 2026]. O conjunto reúne 21 aplicações: formulários, listas, buscas, jogos, reprodução de mídia e recursos de voz. A escolha permite relacionar cada ação a um propósito identificável e examinar tanto operações locais quanto comportamentos dependentes de tempo ou de serviços externos. O catálogo Dopa [da Cunha Franco 2026], desenvolvido pelo próprio autor deste TCC, foi utilizado como cenário inicial de integração do protótipo; os exemplos públicos oferecem os casos de estudo apresentados nesta avaliação.

A caracterização considera organização do código, entradas, estado e dependências. Cada exemplo possui um documento HTML de entrada e dois a quatro arquivos-fonte, com 118 a 372 linhas físicas não vazias de HTML, CSS e JavaScript, incluindo comentários. Dependências instaladas, arquivos gerados, mídia e arquivos minificados são excluídos da contagem. Os exemplos podem ser servidos estaticamente e não possuem serviço próprio no servidor. Cinco utilizam chamadas `fetch`, quatro utilizam armazenamento no navegador e dois empregam APIs de fala.

Essas características descrevem aplicações de pequeno porte e escopo funcional delimitado. O porte é sustentado pela estrutura concentrada no navegador e pela quantidade restrita de responsabilidades, além do volume de código. A exigência de teste, por sua vez, depende do comportamento: uma interface de poucos arquivos pode envolver cálculo, estado acumulado, áudio ou temporização. Por isso, a análise apresenta as funções exercitadas e os resultados observáveis, sem utilizar linhas de código como uma escala de dificuldade.

A Tabela 1 apresenta as aplicações por finalidade predominante. As famílias organizam a leitura dos casos; capacidades como persistência e dependência de rede podem aparecer em mais de uma família. O inventário complementar associa cada aplicação a seus arquivos, medidas, comportamentos de referência e artefatos de execução.

3.3. Exploração e coleta de evidências

Na exploração inicial, o modelo recebe documentação e trechos selecionados de código, estado estruturado da interface e, quando habilitadas, imagens da página. Uma aplicação é considerada desconhecida para a sessão quando seu projeto é apresentado sem sequência de passos ou descrição de defeito a procurar; a definição se refere ao contexto fornecido durante a avaliação. O E2P disponibiliza controles observados em um catálogo de ações.

Tabela 1. Aplicações utilizadas e comportamentos de interesse para exploração.

Família (n)	Aplicações e finalidade	Exigência de observação
Dados e formulários (5)	DOM Array Methods: listas; Expense Tracker: lançamentos; Form Validator: campos; Memory Cards: cartões; Movie Seat Booking: assentos.	Entrada, validação, alteração de valores e estado preservado.
Busca e seleção (5)	Exchange Rate: moedas; Infinite Scroll Blog: publicações; Lyrics Search: letras; Meal Finder: receitas; Product Filtering: produtos.	Consulta, resultados, seleção de detalhe e resposta de serviços.
Mídia e voz (4)	Custom Video Player: vídeo; Music Player: áudio; Speak Number Guess: reconhecimento; Speech Text Reader: síntese.	Controles, identidade do conteúdo e sinais de reprodução ou fala.
Jogos e temporização (5)	Breakout: colisões; Hangman: palpites; New Year Countdown: calendário; Relaxer App: respiração; Typing Game: digitação.	Teclado, sequências, pontuação e mudanças temporais.
Interação e ordenação (2)	Modal Menu Slider: painéis; Sortable List: lista reordenável.	Abertura, fechamento e alteração da ordem.

O modelo escolhe uma operação e seus parâmetros; depois da execução, uma nova observação registra a mudança e orienta a próxima decisão. A Figura 1 apresenta o processo.

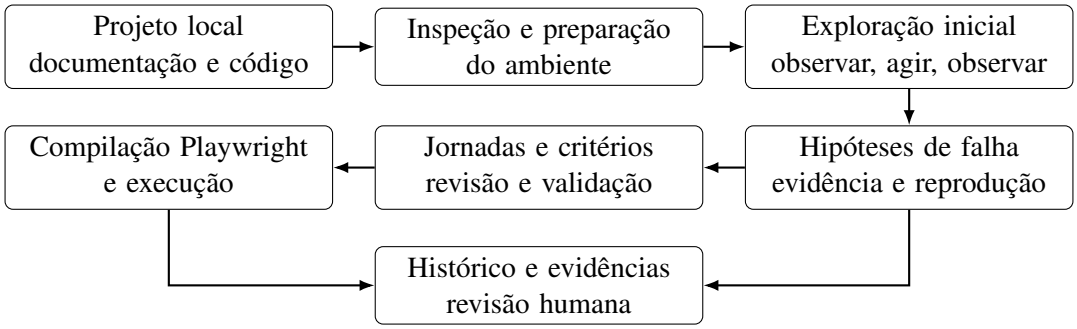


Figura 1. Exploração inicial e transformação das observações em artefatos revisáveis.

A validação inclui duas explorações por aplicação em Expense Tracker, Movie Seat Booking e Modal Menu Slider, alternando os alvos entre repetições. Os casos permitem investigar entrada tipada e cálculo, seleção com persistência e ciclo de painéis, respectivamente. O perfil de acesso é convidado, com dados fictícios e contexto de navegador isolado em cada tentativa. A implementação dos serviços é identificada por resumos criptográficos (*hashes*) e mantida fixa durante a coleta.

Uma rodada complementar utiliza Form Validator, Music Player, Memory Cards e TodoMVC React [TasteJS 2026], com duas repetições intercaladas e o mesmo perfil de modelo. Os primeiros ampliam os comportamentos examinados; o último acrescenta outra origem, componentes React, gerenciamento de estado e filtros por rotas, sem serviço próprio no servidor. Continua sendo uma aplicação pequena, mas exige acompanhar uma coleção entre diferentes apresentações. Seus arquivos-fonte e os serviços também permanecem fixos e identificados durante a coleta.

O procedimento preserva ações, parâmetros, observações, estados e diagnósticos.

Na análise, *exercitar* uma função requer executar sua operação e observar o efeito; *verificar* exige ainda confrontar o efeito com uma expectativa fundamentada. Selecionar um assento, conferir o total e testar restauração são atividades diferentes. A comparação de casos considera essas unidades, sem transformar contagens de passos em cobertura funcional.

3.4. Ambiente de inferência e recursos disponíveis

O modelo local de 27 bilhões de parâmetros utiliza os pesos `Qwen3.8-27B-UD-IQ1_M.gguf`, quantização IQ1_M e projetor visual BF16, sob o nome de configuração `qwen3.8-vl-27b-iq1m-64k`. O servidor llama.cpp b10819 opera com Vulkan, uma posição de atendimento (*slot*) para uma requisição de inferência por vez, contexto configurado de 65.536 tokens, cache de chaves e valores (KV) em Q8 e *Flash Attention* [ggml-org 2026]. A máquina utiliza Windows, Ryzen 7 7800X3D, Radeon RX 9070 XT com 16 GB de memória gráfica e 32 GB de RAM. A exploração e os testes são executados por Playwright com Chromium.

A configuração procura acomodar inferência e automação na mesma máquina. A memória gráfica é compartilhada pelos pesos do modelo, cache de contexto e processamento visual; a RAM também atende o servidor, os processos da aplicação e o navegador. A quantização reduz o espaço ocupado pelos pesos e viabiliza executar esse perfil localmente. A janela configurada estabelece a capacidade máxima de contexto por chamada; o volume efetivamente utilizado depende da etapa e da seleção de evidências.

O orçamento da exploração é de até 20 ações e 180 segundos, com decisões e observações sucessivas. Assim, a profundidade depende tanto da qualidade das escolhas quanto do tempo de inferência, da resposta da aplicação e do custo de organizar o contexto. Selecionar evidências relevantes é parte do funcionamento do instrumento: código, histórico e imagens concorrem por espaço e processamento. O relógio da página é isolado durante decisões do modelo para reduzir a interferência da latência em interfaces temporizadas.

3.5. Validação e critérios de interpretação

A exploração é examinada por ações concluídas, funções reconhecidas, mudanças de estado e diagnósticos observados. A avaliação de um caso executável confronta a intenção do critério com sua asserção: entidade, valor esperado, comparação e momento de verificação. Um teste de persistência, por exemplo, precisa criar estado, recarregar e conferir o mesmo estado restaurado. A existência de um campo ou de um cabeçalho serve como evidência de interface, mas não substitui uma propriedade de domínio.

A implementação é submetida à suíte interna e a verificações de navegador com versões de referência e cópias modificadas dos alvos. São avaliadas mudança de faixa, atualização de saldo e armazenamento, além da preservação de um valor esperado em formulário de controle com regra explícita de normalização. Em cada caso, a mesma suíte é executada no original e na cópia alterada; o efeito da alteração é conferido separadamente no navegador. Esses ensaios verificam a resposta das asserções, enquanto as explorações examinam as decisões do modelo.

Também são repetidas, em sessões limpas por aplicação, ações associadas a exceções no reprodutor de áudio e nos cartões. Para hipóteses de falha, a interpretação reúne

ação executada, efeito observado, expectativa sustentada pelo projeto e resultado da reprodução. A análise dos candidatos utiliza assistência de IA e inspeção dos artefatos; a origem dessa classificação é considerada nas ameaças à validade.

Os arquivos de acompanhamento registram configuração, identificação dos serviços, resultados e referências às evidências. O comando `evaluate:corpus` reproduz o inventário estrutural; os scripts de validação reproduzem as verificações da instância atual. Essa documentação permite examinar cada conclusão no nível de aplicação, ação e propriedade verificada.

4. Descrição da Implementação

4.1. Serviços e preparação do ambiente

O E2P utiliza Node.js e Express, interface HTML/CSS/JavaScript, Cheerio para inspeção estática e Playwright para observação e execução [Microsoft 2026]. Serviços separados tratam inspeção, ambiente, provedor, exploração, planejamento, geração, hipóteses e histórico. A inspeção seleciona documentação e trechos do repositório; o modelo contribui para sua interpretação, enquanto a execução depende de comandos e controles admitidos pelo instrumento. Essa separação torna possível localizar o estágio em que uma falha ocorreu.

O ambiente pode ser servido estaticamente, iniciado por comando, conectado a uma URL existente ou submetido apenas à análise. O comando e a URL inferidos são revisáveis. Cada execução mantém artefatos em diretório próprio, sem inserir testes no código do alvo. Instalar dependências e iniciar a aplicação ainda pode produzir efeitos no ambiente; a separação de diretórios não constitui isolamento completo.

O adaptador atende diferentes provedores; as explorações avaliadas usam llama.cpp diretamente. No modo convidado, capturas podem complementar o modelo de objetos do documento (*Document Object Model*, DOM) quando o provedor suporta visão. O envio ao modelo é controlado separadamente do registro da imagem. O modo autenticado é uma prova de conceito somente para leitura, com referências a segredos por variáveis de ambiente, sessões isoladas e quarentena de artefatos; vídeos e rastros ficam desabilitados nesse modo.

4.2. Exploração limitada e representação de estado

A cada observação, o explorador constrói um catálogo de controles com identificadores, nomes acessíveis e operações admitidas. O modelo escolhe nesse catálogo, e o executor resolve o controle correspondente. Respostas estruturadas em JSON e decisões inválidas recebem correção limitada. A política de participação obrigatória do modelo (*AI-first*) interrompe a progressão quando falta uma decisão válida, preservando a evidência parcial.

Objetivos QA derivados da inspeção contextualizam entrada, seleção, navegação, validação, temporização, persistência e cálculo. O perfil dos registros limita a exploração a até 20 ações e 180 segundos; esperas explícitas são limitadas a 12 segundos. Uma recarga depende de indícios de persistência e mudança anterior. Essas oportunidades orientam a descoberta, mas sua ocorrência não é uma asserção de correção.

O relógio da página convidada é pausado durante a inferência e retomado antes da ação, reduzindo mudanças silenciosas de estado enquanto o modelo decide. A inferência

integra o orçamento total; serviços externos mantêm sua própria temporização. A política prioriza entradas fictícias ordinárias, com correção limitada após validação e respeito aos limites declarados pelos controles. O esgotamento do catálogo encerra a exploração das ações permitidas naquela trajetória.

4.3. Planejamento, compilação e execução

Fluxos e critérios devem remeter a controles e transições registrados. O planejamento recebe resumos de ações e estados com identificadores, reduzindo repetição de catálogos extensos no contexto. A revisão permite selecionar e editar critérios. Em lote, são admitidos fluxos fundamentados de confiança média/alta ou, no modo convidado, um teste explicitamente limitado de transição observada, desde que haja ação concluída, mudança, controle estável e resultado citado.

No caminho convidado, o gerador compila a jornada executada em Playwright, no modo `model-journey-compiled`. O modelo escolhe a trajetória e participa da definição de seu significado; o compilador determina a estrutura executável e as verificações disponíveis. A qualidade do teste depende, portanto, do comportamento alcançado e da preservação do critério na tradução para asserção.

Na instância avaliada, um contrato registra a origem da expectativa, a ação, a comparação, a evidência e a identificação da asserção. Critérios editados são revalidados; os não suportados permanecem não verificados. O contrato torna a tradução inspecionável. O executor distingue término operacional, falha de asserção e erro de infraestrutura. O sucesso exige relatório de execução válido, e medidas de cobertura exigem um conjunto de referência definido.

4.4. Hipóteses, reprodução e memória

Um papel autor propõe hipóteses a partir da trajetória; um papel crítico examina ação, expectativa e efeito alegado. Regras determinísticas filtram candidatos e podem registrar exceções do navegador. Os papéis utilizam o mesmo perfil de modelo. Hipóteses elegíveis passam por reprodução em contexto limpo, repetindo ações conhecidas e comparando o efeito. A procedência da expectativa é conferida como requisito adicional para classificar a violação.

O coletor registra diálogos nativos vinculados à ação, permitindo observar mensagens exibidas fora do DOM. Entradas numéricas fictícias exigem um perfil descartável local explícito. Seus valores respeitam os limites e o incremento declarados no HTML; credenciais continuam sujeitas à política de acesso.

O histórico preserva etapas, erros, testes, capturas e hipóteses em `run-log.json` e artefatos associados. A interface permite registrar revisões humanas com justificativa e conservar as decisões anteriores. Até 20 revisões recentes por caminho de projeto podem contextualizar os papéis autor e crítico. Essa recuperação de contexto complementa a evidência da execução corrente, sem atualizar os pesos do modelo.

4.5. Instrumentação para avaliação

O módulo de caracterização associa alvos a registros e arquivos, produzindo medidas estruturais, identificação dos serviços e referências às asserções. Comportamentos de

referência anotados pelo avaliador permanecem separados do resultado automático. O suplemento publica protocolos, trajetórias e verificações selecionadas, acompanhados de scripts para reproduzir os ensaios. Esse material permite conferir cada conclusão no nível da aplicação, ação e propriedade avaliada.

5. Resultados e Discussão

5.1. Exploração inicial na instância atual

A avaliação reúne 14 explorações em sete aplicações, organizadas em uma rodada inicial e outra complementar, sob o mesmo perfil de modelo. As trajetórias permitem examinar como ele reconhece controles e inicia operações relacionadas à finalidade do produto. A Tabela 2 apresenta as repetições da rodada inicial; a contagem considera ações concluídas, acompanhadas da descrição de seus efeitos.

Tabela 2. Explorações da instância atual: ações concluídas e estados por repetição.

	Ações		Estados	
Aplicação	R1	R2	R1	R2
Expense Tracker	5	5	4	4
Movie Seat Booking	17	17	18	18
Modal Menu Slider	8	8	4	4

As seis explorações registraram 60 ações concluídas e 52 estados somados entre as tentativas. Os identificadores distinguem observações dentro de cada execução. A conferência dos arquivos confirmou que os serviços permaneceram inalterados durante a coleta.

Nas duas explorações do registro financeiro, o modelo preencheu a descrição *Lunch*, informou 15,00, confirmou o formulário e recarregou a página. O histórico manteve o lançamento e o saldo observado foi 15,00. A trajetória alcançou, assim, criação e restauração de estado de domínio. O catálogo informou os limites numéricos e o incremento aceito pelo campo, permitindo que o modelo escolhesse um valor sintético compatível com a validação nativa.

No seletor de assentos, a trajetória percorreu escolhas de filme e lugares, produzindo a maior quantidade de estados da rodada. No exemplo de menu e diálogo, percorreu o ciclo de abertura, interação com campos e fechamento dos painéis. A comparação evidencia alcances distintos: criação e restauração de um lançamento, seleção entre controles e transições de apresentação. A finalidade da aplicação permite interpretar os passos como operações de uso.

O catálogo e os limites de execução são fornecidos pelo E2P; a escolha da operação e de seus parâmetros cabe ao modelo. O lançamento financeiro ilustra essa divisão: os controles e limites numéricos vêm da observação, e o modelo escolhe o dado e a sequência de submissão e recarga. O resultado caracteriza uma exploração guiada por informações do produto e acompanhada de evidências de execução.

5.2. Exploração em contextos complementares

A Tabela 3 apresenta as oito explorações adicionais, com 38 ações concluídas e 26 estados somados entre execuções. A descrição do alcance associa esses registros às operações realizadas em cada aplicação.

Tabela 3. Rodada complementar: ações e estados por repetição (R1/R2).

Aplicação	Ações	Estados	Alcance observado
Form Validator	2/2	3/3	Preenchimento de Username e submissão por Enter; mensagens dos demais campos.
Music Player	3/3	2/2	Navegação de faixa, controle de progresso e reprodução; exceção capturada.
Memory Cards	9/9	3/3	Entradas, limpeza, recarga e painéis; navegação sem cartão produz exceção.
TodoMVC React	5/5	5/5	Criação de tarefa e alternância entre All, Completed e Active.

No TodoMVC, a tarefa *Buy groceries* aparece após Enter, deixa de aparecer no filtro Completed e retorna em Active. A sequência relaciona entrada, criação e seleção de subconjunto. Ela não exercita edição, conclusão nem restauração após recarga. Nos cartões, preencher pergunta e resposta não comprova a criação: o percurso não concluiu essa operação. Esses contrastes mostram que diversidade de ações, funções alcançadas e correção verificada precisam ser analisadas separadamente.

No formulário, o explorador preenche somente Username, pois a política restringe campos de e-mail e senha. Uma repetição gerou a hipótese de que Enter deveria validar apenas esse campo; a outra não a reteve. Uma conferência no navegador mostrou que Enter e o botão Submit disparam a mesma submissão e as mesmas três mensagens. O código valida o formulário completo nesse evento. Assim, a observação é reproduzível, mas a expectativa de validação individual não é sustentada: o caso requer descarte fundamentado, e não confirmação por similaridade de estado.

5.3. Validação das propriedades observadas

A suíte interna registrada na avaliação contém 152 testes aprovados, abrangendo contratos de decisão, observação, evidência, execução e apresentação dos resultados. Os casos confrontam valor esperado e observado, identidade do campo, correspondência entre ação e estado, ambiguidade do critério e coerência do crítico. A verificação de navegador também confere que a validação nativa e seus valores são observados no perfil descartável.

A Tabela 4 sintetiza quatro propriedades verificadas com a mesma asserção em versões de referência e cópias alteradas. O teste aceita o comportamento esperado e falha por asserção quando a propriedade é violada. No registro financeiro, os testes de saldo e persistência são independentes: desabilitar uma dessas propriedades não faz o outro teste falhar.

O formulário de controle verifica uma propriedade central do contrato. Mesmo quando a trajetória registrada mostra 42, o critério aprovado exige 99; o teste gerado conserva 99 como resultado esperado. A implementação de referência aplica essa regra e

Tabela 4. Discriminação de propriedades pela mesma asserção na referência e na cópia alterada.

Propriedade	Diferença introduzida na cópia	Referência	Alterada
Identidade da faixa	Desativar a ação de faixa anterior	Passa	Falha
Saldo do lançamento	Desativar a atualização do saldo	Passa	Falha
Lançamento após recarga	Desativar o armazenamento local	Passa	Falha
Valor aprovado no critério	Produzir 42 quando o requisito exige 99	Passa	Falha

passa, enquanto a cópia que mantém 42 falha. Esse procedimento confirma que a observação do alvo não substitui automaticamente a expectativa utilizada para avaliá-lo.

A identidade do campo e do estado também é validada. Um campo diferente com o mesmo valor não fundamenta a asserção do controle escolhido, e um estado sem correspondência com a ação não é utilizado como seu resultado. Critérios com alternativas ambíguas permanecem disponíveis para revisão. A contribuição desses mecanismos é tornar explícita a relação entre ação, expectativa e comparação executada.

Para o papel crítico, frases afirmativas que sustentam uma hipótese são aceitas como tal, enquanto justificativas negativas incompatíveis com os campos estruturados são detectadas. Candidatos que repetem o mesmo resultado como esperado e observado são rejeitados por não descreverem uma violação. Essa regra foi reaplicada às hipóteses dos painéis, preservando as trajetórias originais para conferência. A verificação complementa as exigências de ação observada e fonte para a expectativa.

5.4. Observação e reprodução de falhas

Na rodada complementar, ações escolhidas pelo modelo expuseram as duas exceções em ambas as repetições. No Music Player, a atualização de tempo tenta escrever em elementos ausentes, produzindo `Cannot set properties of null (setting 'innerHTML')`; o código relaciona o efeito às referências `currTime` e `durTime`. No Memory Cards, navegar sem cartão produz `Cannot set properties of undefined (setting 'className')`, em acesso a `cardsEl[currentActiveCard]`.

O modelo selecionou as ações que alcançaram os estados defeituosos; o coletor do E2P identificou as exceções; a reprodução e a inspeção relacionaram diagnóstico, trajetória e causa. Essa atribuição permite localizar a contribuição de cada componente na observação da falha e disponibilizar uma sequência executável para investigá-la.

Verificação após a coleta. Encerradas as explorações, o reprodutor foi ajustado para tratar recarga e espera como operações do navegador, em vez de procurar controles DOM correspondentes. A mudança permitiu completar a reprodução dos cartões, anteriormente bloqueada nesse ponto. A conferência posterior reproduziu as quatro trajetórias com diagnóstico em sessões limpas, reutilizando os passos gravados. O suplemento conserva separadamente os registros da coleta e os dessa verificação: o resultado adicional diz respeito ao reprodutor, sem alterar as decisões exploratórias registradas.

Outras modalidades exigem canais de observação próprios. Uma mensagem de validação em diálogo nativo pode não alterar o texto do DOM; por isso o E2P registra o tipo, a mensagem e a ação associada ao diálogo. Uma falha de serviço externo exige verificar a resposta recebida e seu tratamento. Da mesma forma, texto preenchido em uma interface de fala é evidência de entrada, enquanto síntese efetiva depende da observação da API ou da saída audível. Essas distinções orientam a análise da falha e a escolha de uma asserção apropriada.

5.5. Profundidade exploratória e capacidade computacional

A execução local deve ser interpretada como uma configuração conjunta de modelo, quantização, contexto e orçamento. O uso de IQ1_M permite acomodar os pesos junto ao cache de contexto e à automação na máquina avaliada. Essa escolha estabelece a viabilidade operacional do perfil; seu efeito sobre a qualidade das decisões permanece uma questão experimental. Assim, uma dificuldade de exploração observada nesse perfil pode envolver tanto a seleção de evidências e a política de ações quanto a configuração de inferência.

Pesos, cache KV, projetor e processamento visual compartilham a memória gráfica. O contexto reservado expressa capacidade máxima; a profundidade depende do contexto efetivamente utilizado e de seu custo por decisão. Mais memória pode permitir menor compressão dos pesos ou mais evidência por chamada, enquanto menor latência pode permitir mais decisões no mesmo intervalo. Para testar essas possibilidades, é necessário registrar tokens utilizados, memória, latência e funções alcançadas. Uma comparação informativa varia contexto, quantização e máquina separadamente; quando o modelo também muda, a unidade comparada passa a ser a configuração completa.

5.6. Respostas às questões de pesquisa

Quanto à **QP1**, o modelo iniciou sequências relacionadas ao uso do produto: criou e restaurou um lançamento financeiro, selecionou controles e painéis e criou e filtrou uma tarefa. O alcance foi parcial: no formulário, a política restringiu campos; nos cartões, a trajetória preencheu entradas sem concluir a criação. A evidência permite identificar quais operações a exploração inicial alcançou.

Para a **QP2**, as trajetórias alcançaram estados com exceções no reprodutor de áudio e nos cartões, registradas pela instrumentação e reproduzidas a partir das ações. O formulário mostrou a situação complementar: um efeito repetível associado a uma expectativa sem sustentação. O processo permite investigar falhas por caminhos distintos, exigindo evidência do efeito e fundamento para a violação alegada.

Para a **QP3**, os contratos e ensaios preservaram o valor aprovado, a identidade da entidade e a correspondência com a ação; a mesma asserção aceitou a referência e rejeitou a alteração controlada. Para a **QP4**, foi caracterizado o ambiente que tornou a exploração local executável e os fatores que condicionam sua profundidade. A relação entre recursos adicionais e ganho de abrangência permanece como hipótese de avaliação comparativa.

6. Ameaças à Validade

Construção. Ações e estados descrevem atividade registrada; o exercício de uma função depende de executar a operação e observar seu efeito. As funções foram interpretadas a partir das trajetórias e dos artefatos, sem uma matriz exaustiva definida antes da coleta.

Por isso, as contagens apresentadas não estimam cobertura funcional completa. Os ensaios com alterações controladas avaliam asserções sobre jornadas conhecidas, enquanto a descoberta inicial é examinada pelas explorações sem roteiro.

Validade interna. As repetições utilizam o mesmo perfil, com temperatura zero nas decisões estruturadas; trajetórias semelhantes caracterizam essa configuração. Os estados iniciais e os arquivos dos serviços foram controlados, mas rede, temporização e dados externos podem afetar o resultado. A pausa do relógio reduz a interferência da inferência e modifica a temporalidade de uso. A verificação do reprodutor após a coleta permanece separada das explorações originais.

Avaliação e atribuição. A análise dos candidatos foi assistida por IA e inspeção dos artefatos, sem classificação humana independente. Os papéis autor e crítico compartilham o perfil de modelo; sua concordância não mede concordância entre avaliadores independentes. A reprodução confirma um efeito, e a alegação de defeito exige ainda uma expectativa justificável. Sem um conjunto exaustivo de defeitos e sem uma linha de base sob o mesmo orçamento, o estudo caracteriza a atuação do sistema combinado, sem estimar precisão, revocação ou vantagem do modelo sobre outra estratégia exploratória. A recuperação de revisões anteriores é um recurso implementado cujo efeito sobre falsos positivos não foi medido.

Validade externa. Predominam aplicações didáticas de uma origem comum, acrescidas de uma aplicação React de tarefas, em modo convidado e Chromium. O porte e as responsabilidades descritos delimitam a transferência para sistemas com mais módulos, autenticação e persistência de servidor. A configuração inclui um único perfil de modelo quantizado; tamanho do modelo, compressão dos pesos e janela útil não foram variados. Também não foi verificado se os exemplos fizeram parte de seu treinamento: “desconhecida” designa a apresentação inicial à sessão. Esses fatores impedem generalizar o alcance observado para outros modelos ou máquinas sem nova avaliação.

7. Conclusões e Trabalhos Futuros

A contribuição do E2P é conectar a iniciativa exploratória de um modelo local a ações executáveis e a evidências que sustentam sua verificação. As trajetórias demonstraram o início de operações relacionadas à finalidade do produto sem roteiro fornecido e o alcance de estados com exceções reproduzíveis. Os ensaios controlados demonstraram a preservação da expectativa aprovada e a discriminação de alterações pelas asserções. O instrumento torna examinável o percurso entre a escolha da ação e o resultado avaliado.

Em relação aos agentes web discutidos na literatura, o trabalho examina a iniciativa anterior à especificação de uma tarefa; em relação à geração de testes, acompanha a origem e a conservação do oráculo. O caso do formulário evidencia por que explorar, reproduzir e julgar uma violação são atividades complementares. A resposta ao problema de pesquisa é, assim, uma capacidade inicial observada em operações delimitadas, apoiada por mecanismos que permitem conferir os resultados e recusar expectativas sem fundamento.

O próximo passo experimental é comparar o modelo com seletores aleatórios ou orientados à novidade, mantendo catálogo, orçamento e compilador. Uma matriz de funções definida antes da coleta permitiria distinguir operação tentada, exercitada e corretamente verificada. A revisão independente das hipóteses e a ampliação das alterações

controladas complementariam essa comparação, examinando tanto as oportunidades alcançadas quanto a qualidade das conclusões produzidas.

Como continuidade, propõe-se utilizar o **JEV**, da TypeSafe, como componente de decisão e avaliação estruturada [TypeSafe AI 2026]. O modelo recebe estado e perguntas tipadas, retornando respostas e distribuições de probabilidade. No E2P, pode ser investigado para selecionar entre ações catalogadas, avaliar se uma evidência sustenta determinada hipótese e encaminhar casos incertos à revisão humana. A avaliação dessa integração deve comparar concordância com julgamentos humanos, custo, latência e comportamento em casos ambíguos, com limiares definidos para cada decisão.

Outra direção é comparar máquinas e configurações de inferência conforme o procedimento discutido na Seção 5.5, medindo funções alcançadas, estabilidade e evidências de falha. Aplicações com jornadas integradas, diferentes perfis de acesso e persistência no servidor ampliariam os comportamentos investigados.

Declaração de uso de inteligência artificial

A ferramenta Codex, da OpenAI, foi utilizada na revisão e atualização das diferentes seções deste artigo: organização textual, tradução do resumo, conferência bibliográfica, confronto com código e artefatos experimentais e formatação LaTeX. Registros de implementação e avaliação também incluem operações assistidas por IA. A definição do problema e das questões de pesquisa, as escolhas metodológicas e a interpretação das evidências integram a contribuição intelectual pela qual o autor responde; a ferramenta oferece sugestões e realiza operações de apoio nessas atividades.

Referências

Ahmed, S. Y., Feng, S., Bae, C., Barrus, C., and Zhang, X. (2026). SpecOps: A fully automated AI agent testing framework in real-world GUI environments. In *Proceedings of the 48th International Conference on Software Engineering (ICSE)*. https://doi.org/10.1145/3744916.3787778.

da Cunha Franco, H. O. (2026). *Dopa*. Repositório de software. https://github.com/pieceofhell/dopa. Cópia local utilizada na avaliação de 8 set. 2026.

Drouin, A., Gasse, M., Caccia, M., Laradji, I. H., Del Verme, M., Marty, T., Boisvert, L., Thakkar, M., Cappart, Q., Vazquez, D., Chapados, N., and Lacoste, A. (2024). WorkArena: How capable are web agents at solving common knowledge work tasks? arXiv:2403.07718, versão 5. https://arxiv.org/abs/2403.07718.

Ferreira, M., Viegas, L., Faria, J. P., and Lima, B. (2025). Acceptance test generation with large language models: An industrial case study. In *IEEE/ACM International Conference on Automation of Software Test (AST)*. https://doi.org/10.1109/AST66626.2025.00007.

ggml-org (2026). *llama.cpp*. Software. https://github.com/ggml-org/llama.cpp. Acesso em: 13 set. 2026.

Gu, S., Nashid, N., and Mesbah, A. (2026). LLM test generation via iterative hybrid program analysis. In *Proceedings of the 48th International Conference on Software Engineering (ICSE)*. https://doi.org/10.1145/3744916.3764553.

Kang, S., Yoon, J., and Yoo, S. (2023). Large language models are few-shot testers: Exploring LLM-based general bug reproduction. In *Proceedings of the 45th International Conference on Software Engineering (ICSE)*. https://arxiv.org/abs/2209.11515.

Kolthoff, K., Kretzer, F., Ponzetto, S. P., Maedche, A., and Bartelt, C. (2025). GUISpector: An MLLM agent framework for automated verification of natural language requirements in GUI prototypes. arXiv:2510.04791. https://arxiv.org/abs/2510.04791.

Le, C. C., Van, C. D., Vu, T. D., Vu, T. M. P., Phan, H. N., Phan, H. N., and Nguyen, T. N. (2026). TestWeaver: Execution-aware, feedback-driven regression testing generation with large language models. In *Proceedings of the 48th International Conference on Software Engineering (ICSE)*. https://arxiv.org/abs/2508.01255.

Liu, Z., Chen, C., Wang, J., Chen, M., Wu, B., Che, X., Wang, D., and Wang, Q. (2024). Make LLM a testing expert: Bringing human-like interaction to mobile GUI testing via functionality-aware decisions. In *Proceedings of the 46th International Conference on Software Engineering (ICSE)*. https://arxiv.org/abs/2310.15780.

Microsoft (2026). *Playwright: Assertions*. Documentação. https://playwright.dev/docs/test-assertions. Acesso em: 13 set. 2026.

TasteJS (2026). *TodoMVC: React example*. Repositório de software. https://github.com/tastejs/todomvc/tree/master/examples/react. Cópia local identificada no suplemento; acesso em: 20 out. 2026.

Traversy, B. (2026). *Vanilla Web Projects*. Repositório de exemplos. https://github.com/bradtraversy/vanillawebprojects. Cópia local utilizada nas avaliações de set. 2026.

TypeSafe AI (2026). *Jev: Introduction*. Documentação oficial. https://docs.typesafe.ai/introduction. Acesso em: 18 out. 2026.

Wang, J., Huang, Y., Chen, C., Liu, Z., Wang, S., and Wang, Q. (2024). Software testing with large language models: Survey, landscape, and vision. *IEEE Transactions on Software Engineering*. Versão dos autores: https://arxiv.org/abs/2307.07221.

Zhao, Y., Talebipour, S., Baral, K., Park, H., Yee, L., Khan, S. A., Brun, Y., Medvidović, N., and Moran, K. (2022). Avgust: Automating usage-based test generation from videos of app executions. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. https://arxiv.org/abs/2209.02577.

Zhou, S., Xu, F. F., Zhu, H., Zhou, X., Lo, R., Sridhar, A., Cheng, X., Ou, T., Bisk, Y., Fried, D., Alon, U., and Neubig, G. (2024). WebArena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/2307.13854.